

Metodología ágil para TFG/TFM

Table of contents

1	1. Objetivo de este documento	2
2	2. Enfoque general: ágil adaptado a TFG/TFM	2
3	3. Roles en el proyecto	2
3.1	3.1. Alumno/a (equipo de desarrollo)	3
3.2	3.2. Director/a (Product Owner)	3
3.3	3.3. Otros actores (stakeholders)	3
4	4. Artefactos y herramientas	3
4.1	4.1. Backlog de producto	4
4.2	4.2. Milestones (hitos)	4
4.3	4.3. Tablero (Project) en GitHub	4
5	5. Sprints: cómo iterar el trabajo	5
5.1	5.1. Duración	5
5.2	5.2. Estructura de un sprint	5
5.3	5.3. Ejemplo de sprint típico	6
6	6. Fases típicas de un TFG/TFM (vista ágil)	6
6.1	6.1. Fase 1: Arranque y planificación	6
6.2	6.2. Fase 2: Diseño / preparación	6
6.3	6.3. Fase 3: Implementación iterativa	7
6.4	6.4. Fase 4: Evaluación y refinamiento	7
6.5	6.5. Fase 5: Cierre y documentación	7
7	7. Calidad, pruebas y CI (opcional para el alumno)	7
7.1	7.1. Calidad mínima esperada	7
7.2	7.2. Integración continua (CI) como opción	8
8	8. Expectativas de comunicación	8

1 1. Objetivo de este documento

Este documento explica **cómo trabajaremos tu TFG/TFM usando metodologías ágiles** (Scrum/Kanban ligeros) en el contexto de la ETSIIT (UGR).

No define la metodología concreta de tu proyecto (eso irá en tu **documento de trabajo específico**), sino el **marco general**:

- Cómo organizamos el trabajo en **iteraciones**.
 - Qué **artefactos** esperamos (backlog, issues, milestones...).
 - Cómo usamos **GitHub** para gestionar el proyecto.
 - Qué se espera de ti antes, durante y después de cada sprint.
-

2 2. Enfoque general: ágil adaptado a TFG/TFM

En un TFG/TFM no tenemos un equipo grande, sino:

- Una persona que **desarrolla** (tú, el alumno).
- Una o varias personas que **dirigen/guían** (director/a, co-directores).

Por tanto, usamos una versión **reducida y pragmática** de metodologías ágiles:

- Trabajamos en **sprints cortos** (1–2 semanas).
- Mantenemos un **backlog** de tareas (issues en GitHub).
- Planificamos y revisamos el trabajo de forma **iterativa**.
- El objetivo de cada sprint es tener algo **enseñable** (prototipo, módulo, experimento, capítulo de la memoria, etc.).

No buscamos cumplir Scrum “al pie de la letra”, sino quedarnos con lo que aporta valor en un TFG/TFM.

3 3. Roles en el proyecto

En este contexto, los roles se mapean así:

3.1 3.1. Alumno/a (equipo de desarrollo)

- Eres responsable de:
 - Planificar tu trabajo por sprints.
 - Mantener el **backlog** y actualizar el estado de las tareas.
 - Desarrollar código, realizar experimentos, escribir documentación.
 - Preparar las demos/entregables de cada sprint.

3.2 3.2. Director/a (Product Owner)

- Actúa como una especie de **Product Owner**:
 - Ayuda a definir y priorizar objetivos.
 - Da feedback sobre lo entregado.
 - Valida el alcance del proyecto y si se cumplen los objetivos del TFG/TFM.

3.3 3.3. Otros actores (stakeholders)

Según el proyecto, puede haber:

- Usuarios finales.
- Otros profesores o grupos de investigación.
- Colaboradores externos (empresas, ayuntamientos, etc.).

Su presencia e implicación se definirá en tu documento de trabajo específico.

4 4. Artefactos y herramientas

Trabajaremos principalmente con:

- **Organización de GitHub** específica de tu proyecto.
- Uno o varios **repositorios** (código, documentación...).
- Un **Project** de GitHub (tablero Kanban).
- **Issues** y **milestones** para gestionar tareas e hitos.

Los detalles prácticos (cómo crear la organización, repos, Projects, etc.) están en [GitHub & organización](#).

4.1 4.1. Backlog de producto

El **backlog** es la lista de todo lo que vamos a hacer:

- Historias de usuario (si tu proyecto las usa).
- Tareas técnicas (configurar CI, escribir tests, refactorizar código, etc.).
- Tareas de documentación (escribir secciones de la memoria, preparar gráficos).
- Tareas de evaluación (diseñar experimentos, preparar cuestionarios, etc.).

En la práctica:

- Cada elemento del backlog es **un issue de GitHub**.
- Deberían tener etiquetas (**backend**, **frontend**, **documentation**, **research**, etc.).
- Se asignan a **milestones** (hitos) y se van moviendo en tu tablero Project.

4.2 4.2. Milestones (hitos)

Recomendamos definir milestones asociados a fases del TFG/TFM. Ejemplo típico:

- **M1 – Análisis y planificación inicial**
- **M2 – Diseño / arquitectura**
- **M3 – Implementación del MVP**
- **M4 – Evaluación / experimentación**
- **M5 – Cierre y memoria**

Cada milestone tiene:

- Una **fecha objetivo**.
- Un conjunto de issues que deben completarse para considerarlo cumplido.

4.3 4.3. Tablero (Project) en GitHub

Utilizaremos un **Project** (tablero Kanban) para visualizar el estado de las tareas.

- Columnas típicas:
 - Backlog
 - En curso
 - En revisión
 - Hecho

Cada issue se mueve de columna según su estado.

Esto ayuda a ver de un vistazo en qué estás trabajando y qué queda pendiente.

5 5. Sprints: cómo iterar el trabajo

5.1 5.1. Duración

En general:

- Sprints de **1 o 2 semanas** funcionan bien para un TFG/TFM.
- La duración exacta se acuerda al inicio del proyecto (y se puede ajustar).

5.2 5.2. Estructura de un sprint

Cada sprint debería incluir, como mínimo:

1. **Planning** (inicio de sprint)

- Revisar el backlog con el director (cuando sea posible).
- Elegir qué issues se abordarán en el sprint.
- Definir un **objetivo claro** para las próximas 1–2 semanas.

2. **Ejecución**

- Desarrollar código, experimentar, escribir memoria, etc.
- Crear issues si surgen nuevas tareas.
- Mantener el Project actualizado.

3. **Revisión**

- Preparar una **demo o resumen** de lo hecho (pantallazos, prototipo, resultados preliminares, sección de memoria redactada...).
- Comentar con el director:
 - Qué se ha conseguido.
 - Qué ha bloqueado o retrasado trabajo.

4. **Retrospectiva informal**

- Reflexionar brevemente:
 - ¿Qué ha ido bien?
 - ¿Qué no?
 - ¿Qué cambiarás en el siguiente sprint?

No hace falta que esto sea pesado: puede ser un mensaje estructurado o una breve reunión.

5.3 5.3. Ejemplo de sprint típico

Sprint (2 semanas)

Objetivo: tener un prototipo funcional de las pantallas principales.

Tareas (issues):

- Diseñar wireframes de la pantalla de inicio.
- Implementar la navegación básica entre pantallas.
- Crear la estructura básica del proyecto (repositorio, dependencias).
- Documentar las decisiones de diseño en el repositorio de documentación.

Entregable:

- Repositorio con app que navega entre pantallas básicas.
 - Documento con capturas y explicación del flujo.
-

6 6. Fases típicas de un TFG/TFM (vista ágil)

Aunque trabajemos con sprints, normalmente podemos distinguir varias **fases**:

6.1 6.1. Fase 1: Arranque y planificación

- Lectura del enunciado y asignación de TFG/TFM.
- Revisión de documentación inicial y trabajo relacionado.
- Definición del **alcance preliminar** con el director.
- Creación de:
 - Organización de GitHub.
 - Repositorios iniciales.
 - Project y primeros milestones.
- Redacción y/o revisión del **documento de trabajo específico** del TFG/TFM.

6.2 6.2. Fase 2: Diseño / preparación

- Diseño de arquitectura (software, hardware o ambos).
- Diseño de modelos de datos, APIs, flujos, etc.
- Plan de experimentación/evaluación si aplica.
- Inicio de redacción de capítulos de la memoria (no esperes al final).

6.3 6.3. Fase 3: Implementación iterativa

- Desarrollo de un **MVP** (mínimo producto viable) lo antes posible.
- Iteraciones sobre funcionalidades, diseño, rendimiento, etc.
- Tests básicos (unitarios, de integración, manuales).

6.4 6.4. Fase 4: Evaluación y refinamiento

- Pruebas con usuarios, datasets, benchmarks... según el tipo de proyecto.
- Análisis de resultados.
- Refinamiento de la solución (ajustes de rendimiento, usabilidad, etc.).

6.5 6.5. Fase 5: Cierre y documentación

- Consolidación del código (limpieza, documentación interna).
- Redacción final de la memoria y anexos.
- Preparación de la defensa (presentación, demo, etc.).

Estas fases **no son lineales**: se solapan y se revisan, pero sirven como guía para tus milestones.

7 7. Calidad, pruebas y CI (opcional para el alumno)

7.1 7.1. Calidad mínima esperada

Independientemente de que uses CI o no, se espera:

- Código razonablemente estructurado, legible y documentado.
- Algún tipo de **pruebas**:
 - Unitarias y/o de integración (según el tipo de proyecto).
 - Pruebas manuales reproducibles (pasos claros).
- Un repositorio con una mínima organización:
 - `src/`, `tests/`, `docs/`, etc.
 - Un **README** que explique cómo ejecutar el proyecto.

7.2 7.2. Integración continua (CI) como opción

La **integración continua no es obligatoria**, pero se recomienda cuando:

- El proyecto tiene cierto tamaño/ complejidad.
- Se pueden automatizar:
 - Ejecución de tests.
 - Análisis estático (linters).
 - Generación de documentación.

Si decides usar CI:

- Lo habitual es usar **GitHub Actions**.
- Configurar al menos un workflow que:
 - Se ejecute en cada push o pull request.
 - Lance tests y/o linters.

En el handbook (repositorio del propio handbook) sí se usa CI para generar la web, y puedes tomarlo como **ejemplo**, pero **no es requisito** que repliques exactamente esa configuración en tu proyecto.

8 8. Expectativas de comunicación

Para que la metodología ágil funcione mínimamente, es importante:

- Acotar una **frecuencia de contacto** razonable con el director (por ejemplo, cada 2–3 semanas).
- Antes de cada reunión, preparar:
 - Qué has hecho (issues cerrados, demos).
 - Qué problemas has encontrado.
 - Qué te propones hacer en el siguiente periodo.

Una buena práctica es:

- Abrir un issue tipo “Informe de sprint N” donde:
 - Resumas los avances.
 - Enlaces a commits, PRs, capturas de pantalla, etc.
 - Anotes dudas para la reunión.

9 9. Cómo se concreta esto en tu TFG/TFM

Todo lo anterior es el **marco general**.

Para tu proyecto concreto:

- Recibirás un **documento de trabajo** (en este mismo handbook), por ejemplo:
 - `pastillero-doc-trabajo.qmd` para el TFG del pastillero inteligente.
- Ese documento:
 - Detalla la metodología específica (por ejemplo, si se usa DCU, técnicas concretas de evaluación, etc.).
 - Aterriza los requisitos, la arquitectura y el planning de tu proyecto.
 - Se irá actualizando si es necesario (versionado en Git).

Siempre que tengas dudas entre lo genérico (este documento) y lo específico de tu TFG/TFM, **prima lo que se indique en tu documento de trabajo concreto**, o consulta al director.